Vu-Le *et al. Applied Network Science* (2026) 11:2
https://doi.org/10.1007/s41109-025-00747-2

Applied Network Science

RESEARCH Open Access

Using stochastic block models for community detection

The-Anh Vu-Le[1†], Minhyuk Park[1†], Ian Chen[1] and Tandy Warnow[1*]

[†]The-Anh Vu-Le and Minhyuk Park have contributed equally to this work.

*Correspondence:
Tandy Warnow
warnow@illinois.edu
[1]Siebel School of Computing and Data Science, University of Illinois Urbana-Champaign, 201 N. Goodwin Avenue, Urbana, IL 61801, USA

Abstract

A recent study reported by Park et al. (Improved community detection using stochastic block models, Springer, Heidelberg, 2025) in Complex Networks and their Applications 2024 showed that clusterings from three Stochastic Block Models (SBMs) in graph-tool, a popular software package, often had internally disconnected clusters when used on large real-world or synthetic networks. To address this issue, Park et al. (Improved community detection using stochastic block models, Springer, Heidelberg, 2025) presented a simple technique, Well-Connected Clusters (WCC), that repeatedly finds and removes small edge cuts of size at most $\log_{10} n$ in clusters, where n is the number of nodes in the cluster, and showed that treatment of graph-tool SBM clusterings with WCC improves accuracy. Here we examine the question of cluster connectivity for clusterings computed using other SBM software or nested SBMs within graph-tool. Our study, using a wide range of real-world and synthetic networks ranging up to more than a million nodes, shows that all tested SBM clustering methods frequently produce communities that are disconnected, and that graph-tool improves on PySBM. We provide insight into why graph-tool degree-corrected SBM clustering produces disconnected clusters by examining the description length formula it uses, and explore the impact of modifications to the description length formula. Finally, we show that WCC generally provides an improvement in accuracy for both flat and nested SBMs, except for cases where nearly all nodes in the network are in very sparse ground-truth clusters. We also demonstrate that WCC scales to networks with millions of nodes.

Keywords Community detection, Stochastic block models, Edge connectivity, Network science

Introduction

Community detection, also known as graph partitioning, is the problem of taking a graph and partitioning the vertices into disjoint subsets so that each set has properties indicative of being a community. Although there is substantial interest in community detection when the input is a dissimilarity matrix or when the graph is directed or has metadata, our focus here is on the simplest version of community detection, where the input is an undirected simple graph without any metadata. Many approaches to community detection in such a context have been developed, and are surveyed in Fortunato

Springer

© The Author(s) 2025. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

(2010); Harenberg et al. (2014); Fortunato and Hric (2016); Zhao (2017); Javed et al. (2018); Cherifi et al. (2019); Jin et al. (2023).

Among the different approaches to community detection, the use of stochastic block models (SBMs) has received very substantial attention, as surveyed in Lee and Wilkinson (2019); Funke and Becker (2019a); Liu et al. (2025). SBMs are probabilistic graphical models where nodes of a graph are partitioned into “blocks", and where there are numeric parameters defining the probabilities of edges between nodes based on block assignment (and in some cases, also taking other information into account, such as a degree sequence). Given a graph, the best fitting SBM can be sought, thus producing a clustering of the nodes in the graph.

SBMs have been extensively studied from theoretical perspectives (Newman 2016; Young et al. 2017; Abbe 2018; Peixoto 2019; Zhang 2024). Specifically, a great deal of attention has been placed on establishing the conditions under which accurate recovery of the true community structure can be guaranteed with high probability as the number of nodes goes to infinity (Abbe 2018). Many of the established positive results require that the number of communities does not grow too quickly, and some explicitly require the number of communities to be a constant. The theoretical guarantees are not well understood when the number of communities is too large, or the average degree is too small, or some other required property fails.

While significant theory has been established for SBMs, much less is known about the empirical performance of community detection based on SBMs, addressing either cluster quality measures or accuracy on synthetic networks.

Communities have been evaluated using various properties that are expected of “true" communities, including edge-density (i.e., proportion of the possible edges present in a cluster), separability from the rest of the graph, and well-connectedness (Yang and Leskovec 2013). To be well-connected, each community should not have a small edge cut, i.e., it should not be disconnected by the deletion of a small number of edges (Kannan et al. 2004; Traag et al. 2019; Yang and Leskovec 2013).

Despite the importance of edge connectivity, some clustering methods produce poorly-connected clusters, and some even produce internally disconnected clusters. As an example, the Louvain algorithm (Blondel et al. 2008), which is often used to find modularity-based clusterings, has been shown to produce internally disconnected communities (Traag et al. 2019), and because of this problem with Louvain, the Leiden algorithm (Traag et al. 2019) was developed, which is guaranteed to produce connected clusters. Leskovec et al. (2010) also noted that a heuristic that combined METIS (Karypis and Kumar 1998) with MQI (Yang and Leskovec 2013) produced internally disconnected clusters.

Of interest to us is the edge-connectivity of communities produced using SBMs. In Peixoto (2019), it was mathematically proven that community detection using certain SBMs will produce disconnected clusters for the special case where the input network has multiple components, each of which is a small clique. However, the problem of internally disconnected clusters for SBMs is not limited to the extreme (and not realistic) network with a collection of cliques that have no connections to each other: a recent study by Park and colleagues (Park et al. 2025) on a large set of real-world networks that ranged up to millions of nodes found that SBM clustering using graph-tool (Peixoto 2014) under three “flat” models–degree-corrected (Karrer and Newman 2011)

(DC-Flat), non-degree-corrected (Holland et al. 1983) (NDC-Flat), and planted partition (Zhang and Peixoto 2020) (PP-Flat)–frequently produced disconnected communities. Clearly communities that are not connected fail a very basic expectation of a valid community, which makes these clusterings undesirable compared to clusterings where all clusters are connected.

Even those communities that are connected may nevertheless not be well-connected, which depends on the size of the minimum edge cut in the cluster; however, the definition of well-connectedness differs between studies. For example, in Traag et al. (2019), the definition of well-connected was posed as a function of the number of edges in the cut relative to the number of possible edges, while in Park et al. (2024), the definition of well-connected depended on the size of the min cut as a function of the size of the cluster. In Park et al. (2024), they explored the function $log_{10}(n)$, thus deeming a cluster of n nodes to be well-connected only when the size of its minimum edge cut was greater than $\log_{10}(n)$. Using this threshold for a cluster to be well-connected, Park and colleagues (Park et al. 2024) found that the Leiden algorithm (Traag 2019; Traag et al. 2019) optimizing modularity or the Constant Potts Model (CPM), Infomap (Rosvall et al. 2009), Iterative-K-core Clustering (IKC) (Wedell et al. 2022), and Markov Clustering (MCL) (Dongen 2008) all produced poorly-connected clusters under some conditions. Also in Park et al. (2024), the authors presented the Connectivity Modifier (CM), a post-processing technique to improve the edge-connectivity of clusters and demonstrated improved clustering accuracy on a selection of synthetic networks. However, a later study (Park et al. 2025) established that the impact of CM depends on the clustering method. Thus, while (Park et al. 2024) established that CM post-processing improved clusterings computed using Leiden optimizing modularity or the Constant Potts Model, Park et al. (2025) showed that applying CM to clusterings produced using graph-tool's flat SBMs had variable impact, sometimes improving accuracy and sometimes decreasing accuracy.

Park and colleagues (Park et al. 2025) proposed two other techniques: Connected Components (CC) and Well-Connected Clusters (WCC). The CC technique replaces every cluster with its connected components, and so is a very simple technique. WCC processes each cluster independently, iteratively finding and removing small edge cuts (if they exist) until the cluster is well-connected (according to a user-specified bound). Using the same threshold of $\log_{10}(n)$ as for the CM method, Park et al. (2025) observed that post-processing SBM clusterings by CC or WCC often improved accuracy on synthetic Lancichinetti-Fortunato-Radicchi (LFR) benchmark networks (Lancichinetti et al. 2008). Specifically, the authors of Park et al. (2025) concluded that it is beneficial to use WCC with the threshold of $\log_{10}(n)$ with clusterings produced using graph-tool's flat models, using the model that had the minimum description length. They also provided an explanation based on the description length for why flat degree-corrected SBMs produce disconnected clusters, especially on large networks.

However, the study reported in Park et al. (2025) was limited in several ways. Most importantly, the study was limited to clusterings produced by graph-tool using its flat models (degree-corrected, non-degree-corrected, and planted partition), and did not examine hierarchical (nested) models available within graph-tool, which are expected to be robust to the SBM resolution limit (Funke and Becker 2019; Peixoto 2019; Zhang and Peixoto 2020; Zhang 2023), whereby the number of clusters that SBMs can detect

has an upper bound that depends on the network size. Furthermore, alternative SBM software was not tested. Another limitation is that accuracy on synthetic networks was only explored using the Adjusted Rand Index (ARI) (Hubert and Arabie 1985), whereas additional accuracy criteria, such as Adjusted Mutual Information (AMI) (Vinh et al. 2009), are also relevant; furthermore, the study used only a few LFR synthetic networks when other synthetic network generators could also have been considered for better reflections of real-world community structure. The explanation based on the description length for why degree-corrected SBMs produce disconnected clusters was similarly limited, since other models were not examined and other reasons for disconnectivity were not examined. Thus, although (Park et al. 2025) provided insight into SBM clustering and the benefit of using WCC to improve clustering accuracy, its limitations to a subset of the models and software are significant, and require further evaluation under additional conditions.

The purpose of this study is to provide a deeper investigation into the properties of clusterings produced using Stochastic Block Models, and the impact of using either CC or WCC for post-processing, using the same threshold of $\log_{10}(n)$. First, we establish that clustering using PySBM (Funke and Becker 2019a, 2019b) or using nested SBMs within graph-tool also produces disconnected clusters, indicating that this issue is not limited to just graph-tool's flat models. We also establish that graph-tool is superior to PySBM in terms of finding good solutions to its optimization problems, i.e., minimizing the description length. Next, we show that nearly all clusters in SBM clusterings of bipartite networks are disconnected. Based on these findings, we restrict the remainder of the study to the use of graph-tool (both flat and nested models) on non-bipartite networks. We then explore the impact of WCC and CC on clustering accuracy using three different accuracy criteria (NMI (Thomas and Joy 2006) and AMI (Vinh et al. 2009) in addition to ARI), for a large set of synthetic networks generated using two new synthetic network generators, EC-SBM (Vu-Le et al. 2025) and ABCD+o (Kamiński et al. 2023), which are able to produce more realistic networks than LFR. These experiments establish that both CC and WCC improve the ability to recover high-quality clusters, but have reduced accuracy with respect to recovering very poor-quality clusters, such as large clusters that are very sparse, which we show are typically produced by modularity-based clustering. We also extend the exploration of the description length formula and its impact on cluster connectivity. Whereas (Park et al. 2025) provided an explanation based on the description length formula for why degree-corrected flat SBMs tend to be disconnected, the authors did not explore other SBM models. Here, we provide additional exploration of the impact of the modifications to the description length formula using different configurations for the weights of the components in the formula. We also explored cases where the true number of clusters is provided. These experiments provide a more nuanced understanding of why SBMs often produce disconnected clusters, and reveal that the explanation in Park et al. (2025) does not fully explain this phenomenon. Finally, Park et al. (2025) did not explore computational performance, and so here we report runtime data for treatments of SBMs on large real-world networks.

Overall, we present empirical results indicating that SBM clustering, using a variety of models and methods, tends to produce a modest number of communities, which tend to be sparse and are often internally disconnected. Our study also shows that SBM clustering accuracy, and the impact of WCC and CC on this accuracy, depends on how well the

ground-truth community matches these features. Thus, SBMs can provide high overall accuracy when the ground truth community structure is characterized by having a very large fraction of the nodes in very sparse ground-truth communities, and in these cases, applying CC or WCC post-processing may reduce accuracy. However, for networks where the network community structure has dense clusters, then applying WCC post-processing improves accuracy.

Materials and methods

Real-world networks

We used a set of 112 real-world networks: 110 from the Netzschleuder network catalogue (Peixoto 2020) and 2 other networks from Park et al. (2024a) (see Supplementary Materials, Section A for the full list of networks). The smallest of these networks (`dnc`) has 906 nodes, and the largest (`CEN`) contains almost 14 million nodes. The four largest networks, `livejournal`, `orkut`, `bitcoin`, and `CEN`, have at least three million nodes each. Thus, 108 of these networks are small-to-moderate in size (from 906 to around 1.4 million nodes), and four are large (at least 3 million nodes). We use the four large networks in a final experiment evaluating computational performance, and the remaining ones for the other experiments.

Networks were obtained as lists of edges without directionality or weights and preprocessed to remove any self-loops or excessive duplicate edges when present; thus, each network we studied was undirected, unweighted, and simple (i.e., no parallel edges and no self-loops).

Synthetic network generation

We used two methods for generating synthetic networks; Edge-Connected SBM Network Generator (EC-SBM) (Vu-Le et al. 2025) and ABCD+o (Kamiński et al. 2023).

EC-SBM

EC-SBM takes as input a network N and some numeric parameters obtained from a clustering $\mathcal{C}$ of N, and then computes a synthetic network modeled on the input, with the goal of reproducing as well as possible various network and clustering statistics. Specifically, EC-SBM is guaranteed to produce a synthetic network with the same number of clusters and sizes, and attempts to come close to other network statistics such as diameter, local and global clustering coefficient, and degree sequence. It also aims to produce the same minimum edge-cut size as the given input clustering. EC-SBM uses graph-tool SBMs to produce some parts of the network, but modifies the network edges in order to improve the fit to the input parameters. Thus, for different clusterings of a given real-world network, EC-SBM produces different synthetic networks.

As shown in Vu-Le et al. (2025), EC-SBM produces networks that better fit their input networks than graph-tool SBMs based on the same input, and are also better in this respect than other synthetic networks, such as LFR (Lancichinetti et al. 2008) and RECCS (Anne et al. 2025). Furthermore, the fidelity between the synthetic network and the real-world network depends on the input clustering. The highest fidelity was obtained when the input clustering was SBM+WCC, where SBM refers to the flat SBM model that had the lowest description length from graph-tool and the second highest fidelity was obtained using Leiden-Mod+CM as the input clustering (Vu-Le et al. 2025).

Since we were evaluating the accuracy of clusterings using SBMs based on software other than graph-tool alone, and we also wished to evaluate the impact of WCC treatments, to avoid favorable results, we did not select EC-SBM networks based on input clusterings that were postprocessed by WCC or CM treatments. Therefore, we selected four clustering methods for use with EC-SBM network generation. We picked one that provided the best fit to the real-world network statistics of these clustering methods when given as input to EC-SBM: Leiden-CPM(0.1). We also picked one that had the worst fit (Leiden-Mod), and the remaining two that had intermediate fit (Leiden-CPM(0.01) and SBM+CC), where SBM refers to the flat SBM model having the lowest description length (i.e., "Chosen-Flat").

Thus, we used EC-SBM to generate 296 synthetic networks, each based on one of 4 different clustering methods and one of the 74 real-world non-bipartite networks described in the previous section. These networks range in size from around 1, 000 nodes (`dnc`) to slightly over 1.4 million nodes (`hyves`).

ABCD+o

ABCD+o takes as input a degree sequence, a community size sequence, a mixing parameter (i.e., the proportion of edge crossing communities), and a number of outliers. These parameters can be obtained from a clustering $\mathcal{C}$ of a network N (with details left to the Supplementary Materials, Section B). We used ABCD+o to generate 148 synthetic networks, each based on one of two different clustering methods Leiden-CPM(0.1) and Leiden-Mod (i.e., the same ones as described above), and one of 74 real-world non-bipartite networks described in the previous section. As these ABCD+o networks sometimes had internally disconnected clusters, we replaced each such cluster by its connected components (i.e., a post-processing using CC).

Clustering using SBMs

For an input network, we generated SBM clusterings using three different approaches.

- We used graph-tool to produce a "chosen" SBM clustering, with the following protocol. First, we clustered the network using three different flat SBM models: DC-Flat, NDC-Flat, and PP-Flat. We then selected the clustering that achieved the lowest description length. This clustering is referred to as the "Chosen-Flat" clustering of the input network.
- We used graph-tool to produce a "chosen" hierarchical (nested) SBM clustering, with the following protocol. First, we clustered the network using two different nested SBM models: the degree-corrected nested model (DC-Nested) and the non-degree-corrected nested model (NDC-Nested). We then selected the model with the lower description length. Finally, the bottom level (i.e., most refined) of the selected model was returned as clustering. The model (DC-Nested or NDC-Nested) having the lower description length is the "Chosen-Nested" clustering of the input network.
- We used the PySBM package (Funke and Becker 2019), which generates SBM clusterings under a wide range of models. PySBM requires that the number of blocks be specified by the user. We set this number using the number of blocks computed using graph-tool on the same network; see Supplementary Materials, Section C for details.

Details of the software and commands used to generate SBM clustering for all approaches are provided in the Supplementary Materials, Section C.

Post-processing treatments to improve connectivity

We post-processed clusters in two ways (below), each of which was designed to improve the edge-connectivity of the clusters in a given clustering. The software and commands used to post-process the clusters are detailed in Supplementary Materials, Section D.

Connected Components (CC)

Given a cluster that is internally disconnected, we replaced it with its connected components; thus, each single disconnected cluster is replaced by two or more smaller clusters that are connected.

Well-connected clusters (WCC)

The WCC technique is a simplification of CM (Park et al. 2024a) that does not re-cluster during the iterative process. Thus, given a clustering of a network and a threshold for well-connectedness (defined by the minimum edge-cut size), WCC checks each cluster to see if it is well-connected. If so, it places the cluster in the output; else, it partitions the cluster into two smaller clusters based on its minimum edge cut. The process iterates until each cluster is well-connected.

As with CM, we used the default threshold from Park et al. (2024a) for well-connectedness, which requires a cluster to have a minimum edge cut size that is strictly greater than $\log_{10}(n)$, where n is the number of nodes in the cluster.

To reduce the runtime, if the minimum edge-cut could be obtained by deleting a single edge that separates one node from the rest of the cluster, we used the partition produced by removing that edge. Otherwise, the minimum edge-cut was the balanced minimum edge-cut computed by the VieCut (Henzinger et al. 2018) software with the cactus data structure.

Evaluation

We evaluated clusterings produced by the SBMs with and without post-processing using CC and WCC on both real-world and synthetic networks.

For a real-world network, we analyzed the connectivity of the estimated clusters, noting the proportion of clusters that are disconnected, poorly connected, and well-connected. We also evaluated methods for SBM clustering under different models based on the description length they find (lower is better). The code for the computation of the description length is provided in Supplementary Materials, Section E.

For a synthetic network with ground-truth communities, we evaluated clustering accuracy using NMI, ARI, and AMI. We also computed clustering accuracy after restricting the network to a subset of the nodes based on membership in ground-truth clusters that had sufficient edge-density. Specifically, for a density threshold t, we include a cluster C if the number of edges in C is greater than $t \binom{n}{2}$, where n is the number of nodes in C. We consider a singleton cluster to have density 0.0. Finally, we also evaluated estimated clusterings using precision and recall, so that each ground-truth cluster or estimated cluster defines a set of pairs, and we compared the set of pairs for estimated

and for ground-truth clusterings. See Supplementary Materials, Section F for additional details.

Infrastructure

This work made use of the Illinois Campus Cluster, a computing resource operated by the Illinois Campus Cluster Program (ICCP) in conjunction with the National Center for Supercomputing Applications (NCSA).

Each method was allowed up to 72 h of runtime, 256GB of RAM, and 16 cores of parallelism. If a clustering method failed to complete on the network, we reported this and provided the reason for the failure.

Experiments

We conducted five experiments:

- Experiment 1: We compared PySBM to graph-tool to determine the best approach for SBM-based clustering. Based on this experiment, we selected graph-tool for future study.
- Experiment 2: We evaluated edge-connectivity of clusters produced by graph-tool SBM clusterings, both flat and nested, on real-world networks.
- Experiment 3: We evaluated the impact on clustering accuracy of our two treatments on SBM clusterings using graph-tool, both flat and nested, on synthetic non-bipartite networks with ground-truth community structure.
- Experiment 4: We analyzed the influence of different components in the description length formula for DC-Flat in graph-tool.
- Experiment 5: We evaluated the computational performance of our treatments on large real-world networks.

Results

Experiment 1: Comparing PySBM to graph-tool

This experiment compared PySBM (Funke and Becker 2019) to graph-tool on real-world networks with respect to the proportion of disconnected clusters and the ability to find clusterings with the minimum description length for a specific SBM model. The experiment was conducted on the 10 smallest real-world networks, with the number of nodes between 906 and 2115 (see the list in Supplementary Materials, Section G. PySBM implements three inference algorithms for minimizing description length: Kernighan-Lin (KL-EM) (Kernighan and Lin 1970), Metropolis-Hastings with 250,000 iterations (MHA-250k), and Peixoto's Agglomerative Heuristic (PAH) (Peixoto 2014). Note that PAH is similar to the main inference technique implemented in graph-tool.

Figure 1 shows the results for two SBM models that are implemented in both graph-tool and PySBM: degree-corrected (DC-Flat in graph-tool, DCPUH in PySBM) and non-degree-corrected (NDC-Flat in graph-tool, SPC in PySBM). We compared graph-tool's algorithm with the three PySBM's algorithms. The top row shows the percentage of clusters that are internally disconnected in the computed clustering. For the bottom row, we treated the description length of the clustering computed by the graph-tool's algorithm as the baseline and report the relative description length score, defined as the ratio of the description length of the clustering obtained by each inference algorithm to the baseline. Hence, the score for graph-tool's algorithm is set to 1.0, and the score for PySBM's

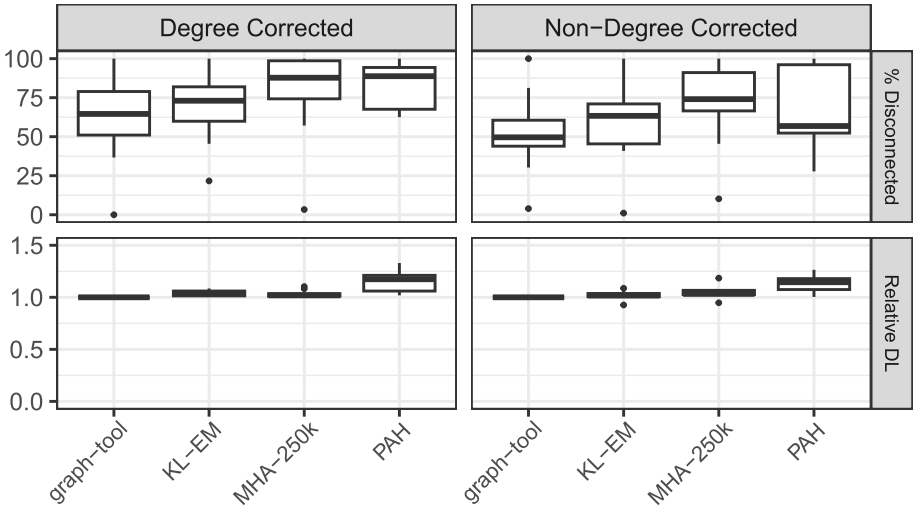


Fig. 1 Experiment 1: Comparing PySBM and graph-tool We compared three algorithmic techniques implemented in PySBM (KL-EM, MHA-250k, PAH) and graph-tool on the ten smallest real-world networks (between 906 and 2115 nodes in each network). The PySBM models use the number of blocks from graph-tool. Left: results for degree-corrected; right: results for non-degree-corrected; top: percent of clusters that are disconnected; bottom: relative description length (DL) values compared to those found by graph-tool (values greater than 1.0 indicate worse results compared to graph-tool)

inference algorithms could be larger or smaller than 1.0: values less than 1.0 indicate that PySBM found a model with a better (lower) description length, and values greater than 1.0 indicate that PySBM found a model with worse (higher) description length.

For both models studied (DC and NDC), although graph-tool and the best of PySBM's algorithms (KL and MHA-250k) had close description length scores, there was a slight advantage to graph-tool. PAH had worse performance for description length. In addition, for both models, graph-tool clusterings had a smaller percentage of disconnected clusters than those found by any of PySBM's techniques.

PySBM also implements additional SBM models beyond the two in common with graph-tool. In Supplementary Materials, Fig A we see that all of these models produced a high frequency of disconnected clusters.

Based on these observations, we restricted the rest of the study to using graph-tool.

Experiment 2: Evaluating edge-connectivity of graph-tool clusterings on real-world networks

Table 1 shows the percentage of clusters that were internally disconnected in clusterings produced by five of graph-tool's SBM models on the 108 small to moderate-sized real-world networks (see Materials and Methods).

On both bipartite and non-bipartite networks, a high percentage of the clusters were internally disconnected. Moreover, the percentage was much higher for bipartite networks than for non-bipartite networks. Based on these observations, we restricted the rest of the study to the networks that are not bipartite.

On the non-bipartite networks, all models produced at least 62.5% disconnected clusters, with the highest percentage produced by DC-Flat and the lowest by PP-Flat. An in-depth examination of the connectivity of these clusterings is provided in Supplementary Materials, Figs B and C, which suggests that the incidence of disconnected clusters may

Table 1 Experiment 2: Percentage of clusters that are disconnected for SBM models on real-world networks

	Bipartite (34)		Non-Bipartite (74)	
Model	**Mean**	**Std. Dev.**	**Mean**	**Std. Dev.**
NDC-Flat	100.0	0.0	68.8	23.1
DC-Flat	100.0	0.0	76.1	23.8
PP-Flat	86.9	22.8	62.5	30.8
NDC-Nested	100.0	0.0	67.6	22.5
DC-Nested	100.0	0.0	72.4	24.6

We show the mean and standard deviation of the percentage of internally disconnected clusters across 34 bipartite networks (left) and 74 non-bipartite networks (right) for 5 different SBM models implemented in graph-tool

increase as the network size increases, and that some models (e.g., PP-Flat) may tend to have more poorly-connected clusters than other models.

Experiment 3: Impact of treatment on synthetic networks

Experiment 3 has two sub-experiments. In Experiment 3a, we evaluated the overall impact of the CC and WCC treatments on clustering accuracy for SBM clusterings (both flat and nested) on EC-SBM and ABCD+o networks. Experiment 3b then examined this impact in depth, examining how using CC and WCC improved or hurt recovery of ground-truth clusters, as a function of the cluster density. Recall that CC replaces a cluster that has multiple components by the collection of components, while WCC makes potentially more substantial changes. Specifically, given a cluster, its minimum edge cut is computed, and if the size is larger than $\log_{10}(n)$ (where n is the number of nodes in the cluster), then the cluster is considered "well-connected" and is put in the output bin; otherwise, the cluster is split into two parts based on the computed cut, and both parts are recursively analyzed.

Experiment 3a: Evaluating overall impact of treatments on clustering accuracy

Experiment 3a evaluated the impact of the CC and WCC treatment on Chosen-Nested and Chosen-Flat SBM clusterings produced using graph-tool, for all four types of EC-SBM networks and two types of ABCD+o networks, as defined by the input clustering given to EC-SBM or ABCD+o to define the parameters for the simulation.

The impact of the CC and WCC treatment on Chosen-Nested or Chosen-Flat is shown in Table 2. The only cases where CC or WCC hurt accuracy were for those networks based on Leiden-mod; otherwise CC and WCC improved accuracy for all three criteria. For ARI and NMI, WCC resulted in a larger improvement than CC for networks based on Leiden-CPM(0.1) or Leiden-CPM(0.01), and CC resulted in a larger improvement than WCC for networks based on SBM+CC. For AMI, CC resulted in a larger improvement than WCC on networks based on Leiden-CPM(0.1), Leiden-CPM(0.01), and SBM+CC networks. However, whenever the synthetic network was based on the Leiden-Mod clustering, then these treatments reduced accuracy for all three criteria.

Understanding why CC and WCC have variable impact To understand why the impact of CC and WCC depends on the type of EC-SBM network, we investigated the properties of the ground-truth clusterings and computed clusterings of the EC-SBM networks. For each of the four types of EC-SBM networks, we computed the size and edge-density (i.e., fraction of the edges that are present) for the clusters. After binning the clusters by edge-

Table 2 Accuracy of methods with and without treatments

	Base	Treatment	AMI	ARI	NMI
EC-SBM types					
Leiden-CPM(0.1) (73 networks)	Chosen-Flat	None	0.3088	0.0152	0.5821
		+CC	0.4240	0.0418	0.8025
		+WCC	**0.4567**	0.1496	0.8907
	Chosen-Nested	None	0.2471	0.0052	0.5206
		+CC	0.4372	0.1116	0.8794
		+WCC	0.4187	**0.1900**	**0.9100**
Leiden-CPM(0.01) (73 networks)	Chosen-Flat	None	0.4327	0.0406	0.5333
		+CC	**0.5227**	0.1639	0.8156
		+WCC	0.4980	**0.2540**	**0.8616**
	Chosen-Nested	None	0.3558	0.0265	0.5209
		+CC	0.4308	0.1199	0.8386
		+WCC	0.3730	0.1487	0.8551
SBM+CC (74 networks)	Chosen-Flat	None	0.4528	0.1904	0.6591
		+CC	**0.8115**	**0.6694**	**0.9468**
		+WCC	0.6160	0.4037	0.9118
	Chosen-Nested	None	0.4430	0.1244	0.6558
		+CC	0.6037	0.3975	0.9207
		+WCC	0.4722	0.1950	0.8901
Leiden-Mod (71 networks)	Chosen-Flat	None	**0.7119**	**0.4671**	**0.7605**
		+CC	0.4237	0.3113	0.5812
		+WCC	0.2777	0.1525	0.5264
	Chosen-Nested	None	0.6799	0.3823	0.7013
		+CC	0.3165	0.1734	0.5411
		+WCC	0.2297	0.0716	0.5120
ABCD+o types					
Leiden-CPM(0.1) (71 networks)	Chosen-Flat	None	0.0424	0.0000	0.2263
		+CC	0.1458	0.0016	0.6889
		+WCC	0.4536	0.0778	0.9276
	Chosen-Nested	None	0.0508	0.0000	0.2639
		+CC	0.2843	0.0082	0.8816
		+WCC	**0.4544**	**0.1504**	**0.9357**
Leiden-Mod (72 networks)	Chosen-Flat	None	**0.7434**	**0.6907**	**0.7587**
		+CC	0.6482	0.5836	0.7322
		+WCC	0.3658	0.2015	0.6357
	Chosen-Nested	None	0.6888	0.4741	0.7132
		+CC	0.3541	0.1256	0.6544
		+WCC	0.2141	0.0481	0.5893

Each value is the median across all networks. Underlined values indicate the best treatment for a given base method. Boldfaced values indicate the best for a given network type

density, we observed features that are common across the different types of EC-SBM networks as well as striking differences (see Fig. 2).

Most importantly, the ground-truth clusterings of the EC-SBM networks based on Leiden-Mod clusterings differed substantially from the other types of EC-SBM networks: they had the vast majority (median value close to 100%) of the nodes in the cluster density bin (0.0,01) (Fig.2), and these very low density clusters were very large (see Supplementary Materials, Fig D). In contrast, EC-SBM clusterings based on the other clusterings ranged from close to 0% to at most 45% for median percentage in these sparsest clusters. We also observe that EC-SBM networks based on Leiden-CPM(0.1) clusterings had very few clusters in the lowest density bin, and the bin with the largest number of nodes was relatively dense (i.e., edge-density between 0.3 and 0.4). The

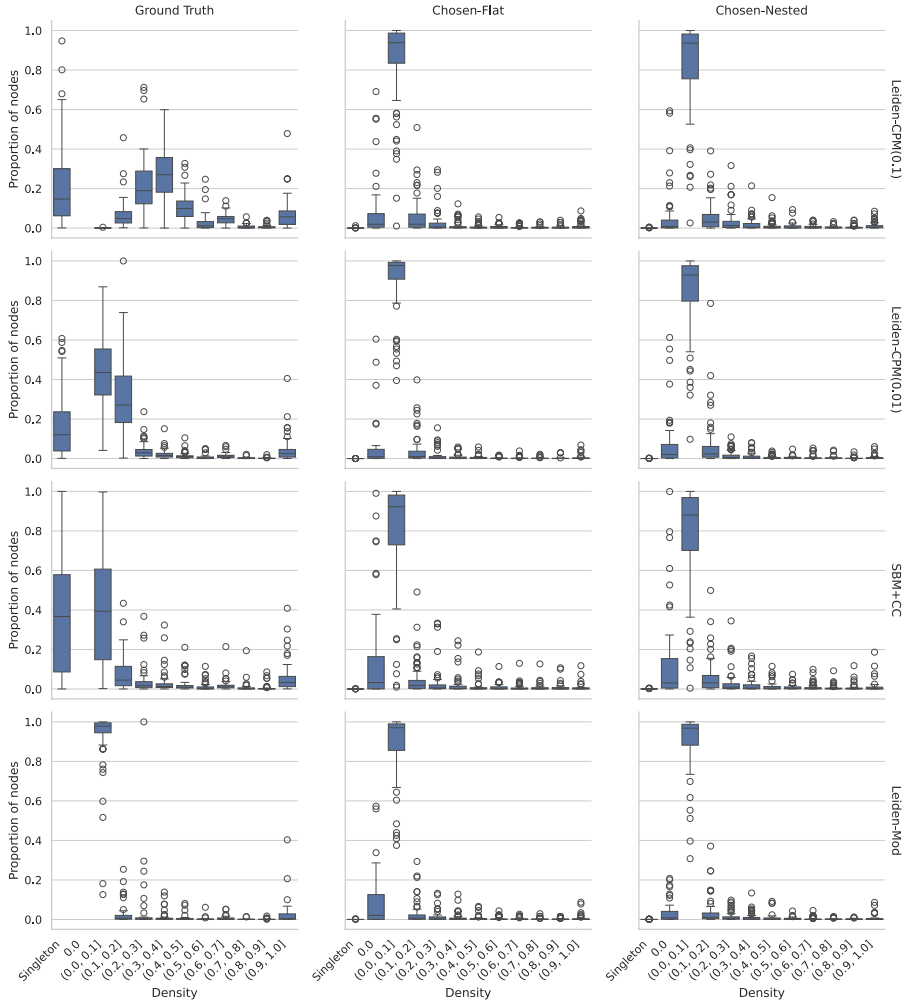


Fig. 2 Proportion of nodes in clusters, binned by density, for ground-truth and estimated clusterings Each row corresponds to a set of 74 EC-SBM networks that are defined by the input clustering method given to EC-SBM, specified in the row label. The clusters of each network are binned into sets based on density. Each boxplot shows the distribution across all networks of the proportion of nodes in clusters in each density bin

EC-SBM networks based on Leiden-CPM(0.1) clusterings also had the largest median fraction of nodes in the highest density bin. Perhaps related to this large fraction of nodes in the sparest clusters, we note that only EC-SBM networks based on Leiden-Mod had essentially no unclustered nodes; in contrast, all other EC-SBM network types had substantial fractions of unclustered nodes (singleton clusters, equivalently), with medians ranging from about 10% to nearly 40%.

With this context, we now examine the density distribution of the computed clusterings, Chosen-Flat and Chosen-Nested, on EC-SBM networks. Note that for all four EC-SBM network types, the vast majority of the nodes (medians of at least 92%) are in the sparsest clusters (see Fig. 2). Thus, the SBM clusterings on these 74 synthetic networks resemble Leiden-Mod clusterings rather than the Leiden-CPM or SBM+CC clusterings, in terms of the density distribution.

An examination of the number of clusters for both ground-truth and computed clusterings is shown in Table 3. Here we see that both Chosen-Flat and Chosen-Nested computed SBM clusterings generally produce fewer clusters than the ground-truth

Table 3 Total number of clusters of ground-truth and estimated clusterings

	Method	Min	Q1	Med	Q3	Max
Leiden-CPM(0.1)	Ground Truth	28	587.50	2565.50	14050.25	150433
	Chosen-Flat	1	29.00	72.50	249.75	5864
	Chosen-Nested	5	39.25	117.50	396.00	23134
Leiden-CPM(0.01)	Ground Truth	1	245.75	858.00	4963.50	54620
	Chosen-Flat	2	25.25	81.00	220.25	2598
	Chosen-Nested	3	41.50	122.00	345.50	18658
SBM+CC	Ground Truth	23	143.00	525.00	3305.00	78810
	Chosen-Flat	2	39.00	87.00	377.00	2994
	Chosen-Nested	5	54.00	138.00	543.00	17315
Leiden-Mod	Ground Truth	3	26.00	71.50	334.00	15738
	Chosen-Flat	3	23.25	42.00	94.25	397
	Chosen-Nested	7	32.50	58.50	112.75	1818

Each network type has 74 networks that are defined by the input clustering method given to EC-SBM. When computing the number of clusters, singleton clusters are excluded (their nodes are considered unclustered)

clusterings, with Chosen-Flat producing fewer than Chosen-Nested. The ground-truth clusterings that have the highest density also have the largest number of clusters, so that the gap between the correct number of clusters and what Chosen-Flat and Chosen-Nested selected is largest for EC-SBM networks based on Leiden-CPM(0.1) clusterings and smallest for EC-SBM networks based on Leiden-Mod clusterings. Finally, for the EC-SBM networks based on Leiden-Mod clusterings, the gap is relatively small: the median number of clusters produced by Chosen-Flat is 59% of the correct value, and the median number of clusters produced by Chosen-Nested is 82% of the correct value. Thus, although Chosen-Nested produces fewer clusters than the true number, it nevertheless more closely resembles the ground-truth clusterings for EC-SBM networks based on Leiden-Mod in terms of the number of clusters than other types of EC-SBM networks.

Thus, the EC-SBM networks based on different input clusterings differ substantially in terms of the number of ground-truth communities and the density and size of these clusters. While it is not clear exactly how each of these differences contributes to the impact of WCC, one obvious explanation is simply this. graph-tool's SBM favors the most parsimonious model that can explain the structure of the network and its community by minimizing the description length. When the number of ground-truth communities is large, the cost of describing the more complex true clustering may outweigh the benefit of a better fit. This is known as the underfitting problem, or the resolution limit (Peixoto 2013), and is illustrated in Peixoto (2019); Zhang (2023) through an example of a network with disconnected cliques, where SBM merges some of the cliques together. Consequently, SBM may produce a smaller number of communities than the true number, and each of which will be less dense than the true communities, like what we observed here. Using WCC on these clusters will tend to break them up into smaller and denser communities, thus increasing the number of communities and potentially improving the clustering accuracy. In contrast, when the number of true communities is close to the number that the SBM is able to return, then SBM clustering may produce a relatively accurate clustering. If that clustering contains poorly-connected clusters, then applying WCC will break up those clusters, thus potentially reducing accuracy. This explanation fits with the observation in this experiment where WCC was overall detrimental for EC-SBM networks based on Leiden-Mod, and otherwise was either beneficial or neutral.

Experiment 3b: Examining impact of treatment on cluster recovery binned by density

We examined results on EC-SBM networks derived from Leiden-Mod clusterings of the real-world networks in greater detail, since this was the only EC-SBM network type for which CC and WCC post-processing of Chosen-Flat or Chosen-Nested SBM clustering resulted in reduced accuracy. We present results for Chosen-Nested in Fig. 3; results for Chosen-Flat are very similar and are shown in Supplementary Materials, Fig E.

Note that while overall accuracy was reduced for each of the three criteria, accuracy *improves* as the threshold for density of the ground-truth cluster increases. Thus, for example, applying WCC to Chosen-Nested improves recovery of all ground-truth clusters whose density is at least 0.03, which is quite sparse. The minimal density threshold for CC to result in an improvement is even lower: only 0.02. This implies that the reduction in overall accuracy that results from using WCC is due to a reduction in accuracy on the sparsest clusters, rather than a reduction in general.

Moreover, we also see that if ground-truth clusters of density at least 0.1 are considered, the difference between untreated and treated Chosen-Nested is large, as seen in

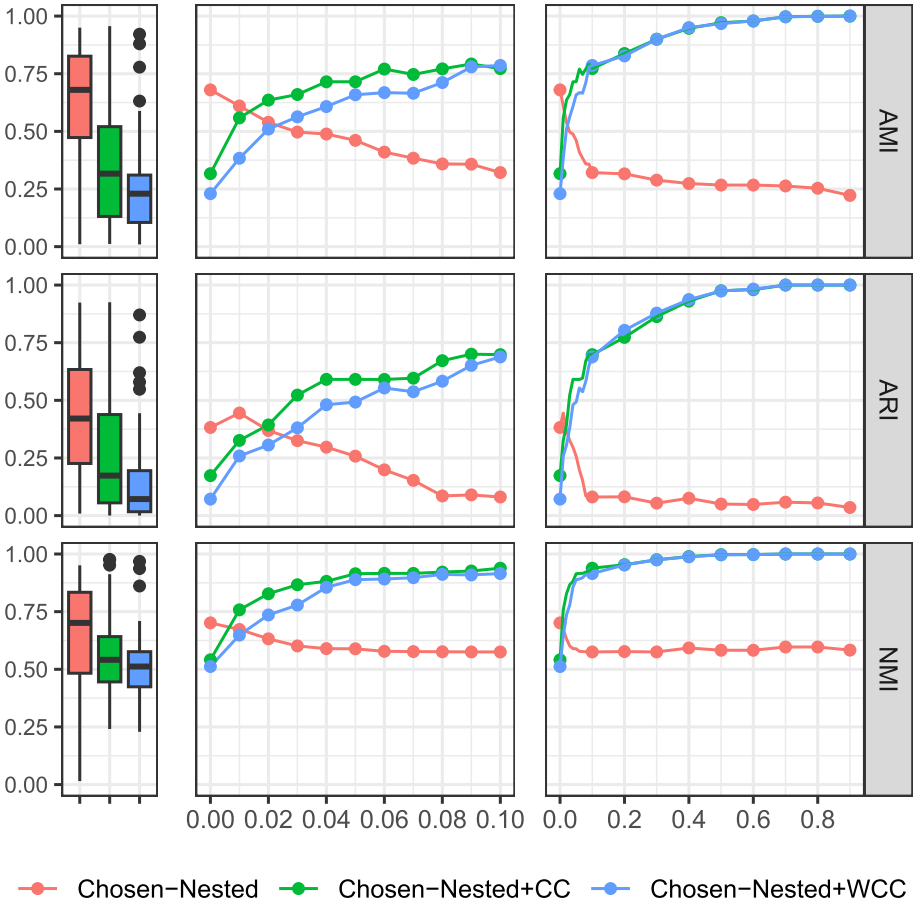


Fig. 3 Experiment 3: Effect of treatments on Chosen-Nested on 71 EC-SBM Leiden-Mod networks for recovery of ground-truth clusters as a function of the minimum density Results are not shown for three networks (`myspace_aminer`, `berkstan_web`, and `petster`) due to memory issues for WCC on these networks. The left column shows the accuracy when all ground-truth clusters in each network are considered. For the middle and right columns, the *x*-axis represents a density threshold: only ground-truth clusters with a density strictly greater than the threshold are analyzed (but singletons are included in the 0.0 threshold); results shown are medians across all the networks. Both CC and WCC treatments generally improve accuracy for Chosen-Nested for clusters with density at least 0.03, but can reduce accuracy for the sparsest clusters (density at most 0.02)

the right panel of Fig. 3. Thus, WCC and CC both have a very positive impact on the ability to recover ground truth clusters if they are *not too sparse*, and this is true even for the EC-SBM network based on Leiden-Mod.

Understanding why untreated SBM clustering has reduced accuracy as cluster density increases We comment on the observation that accuracy for untreated SBM clusterings, both flat and nested, often started high and then decreased as the density threshold increased (see Fig. 3 and Supplementary Materials, Fig F). An explanation for this may be seen by noting that ground-truth cluster size tended to decrease with the cluster density, but Chosen-Flat and Chosen-Nested SBM clusterings of these synthetic networks had very few nodes in small clusters (see Supplementary Materials, Fig D and note that the trends on Chosen-Nested and Chosen-Flat are almost identical). For additional insight, we show precision and recall for Chosen-Flat and Chosen-Nested, with and without WCC treatment, in Supplementary Materials, Fig G. These results show that while recall improved for all methods as density increased, precision tended to decrease for untreated Chosen-Flat and untreated Chosen-Nested, indicating that (to some extent) nodes from different dense ground-truth clusters were being merged into larger clusters. In contrast, precision tended to improve with increases in density when Chosen-Flat and Chosen-Nested are WCC-treated. This improvement in recall and precision for WCC-treated methods also provides an insight into why applying WCC post-processing improved accuracy when restricted to dense clusters: WCC breaks up the large sparse clusters into smaller dense clusters that are closer to the ground-truth clusters.

Experiment 4: Understanding why stochastic block models produce disconnected clusters

In this section, we investigate the reason for disconnectivity in SBMs. In Table 3, we have observed that SBMs tend to underestimate the number of clusters. We analyzed the objective function of the graph-tool optimization problem, the description length of the computed SBM, and identified the priors, specifically the edge count matrix description length, to be a potential cause. This motivated the two following experiments. We first experimented with reducing the influence of the priors by lowering their weights in the description length or deactivating the most influential component. We then experimented with giving graph-tool the ground-truth number of clusters. Both experiments showed potential benefits, but neither resolved the issue completely (i.e., graph-tool still produces disconnected clusters).

Analysis of the DC-Flat description length formula

For a given input network G, graph-tool under the DC-Flat model seeks a clustering that minimizes the description length, making it an optimization problem. We now define the description length for the DC-Flat stochastic block model. Let

- A be the adjacency matrix, which represents the network G,
- b be the block (cluster) assignment, which represents the clustering of G,
- k be the degree vector, defined by A,
- e be the edge count matrix, defined by A and b,
- $\beta \in [0, 1]$ be the weight of the priors (default to $\beta = 1.0$).

Equation (1) provides the formula for the description length $\mathrm{DL}(A, b)$ of a network represented by A and a clustering represented by b under the DC-Flat model:

$$\mathrm{DL}(A, b) = -\log p(A|b, e, k) - \beta \log p(k|b, e) - \beta \log p(e|b) - \beta \log p(b) \tag{1}$$

In this section, we consider the default configuration of $\beta = 1.0$.

Equation (1) can be decomposed into four parts, which are the negative logarithms of the model likelihood ($-\log p(A|b, e, k)$), the prior for the degree sequence ($-\log p(k|b, e)$), the prior for the edge count matrix ($-\log p(e|b)$), and the prior for the block assignment ($-\log p(b)$).

In Peixoto (2019), Peixoto et al. presented an analysis regarding the "resolution limit" of SBM clusterings. The analysis was done under a model with

$$p(e|b) = \frac{\bar{\lambda}^E}{(\bar{\lambda}+1)^{E+B(B+1)/2}} \tag{2}$$

where B, E are the numbers of clusters and edges, respectively, and $\bar{\lambda} = 2E/B(B+1)$. Using this expression of $p(e|b)$, Peixoto (2019) derived that the increase in the number of clusters B will incur a quadratic growth in the description length. Peixoto (2013) showed that, under this model and *assuming that the average degree stays constant*, the maximum number of clusters detectable by SBM will grow with $\sqrt{N}$ where N is the number of nodes in the network. However, we focus our analysis on the case where we have a fixed input network, and therefore a fixed number of nodes and edges.

Since the theory derived in Peixoto (2019) is for a model that is not one of the models we analyze in this study, which are closer to Peixoto (2013), we provide an analysis here for the DC-Flat model. In fact, in Park et al. (2025), we provided this analysis of the formula for the $-\log p(e|b)$ component of DC-Flat. For the sake of completeness, we include it here:

$$-\log p(e|b) = \log \binom{B(B+1)/2 + E - 1}{E} \tag{3}$$

where B is the number of blocks and E is the number of edges. The formula shows that $-\log p(e|b)$ will increase logarithmically as B increases (from 1 to N, where N is the fixed number of nodes). To see this, note that the binomial coefficient is a polynomial in B of degree $2E$. Thus, this favors a small number of blocks B. One interesting note here is that the growth rate is scaled by the number of edges, and so networks with more edges will have a steeper penalty for having more clusters.

Recall that both the CC and WCC treatments tend to increase the number of blocks B, and that even CC increases this number whenever any of the clusters are disconnected. Since increasing the number of blocks makes $-\log p(e|b)$ larger, this means that such treatments will tend to result in larger description lengths—which will not be favored by SBM. Since the description length is impacted strongly by $-\log p(e|b)$, this explains why SBM clustering will tend to prefer clusterings that have internally disconnected clusters rather than their CC-treated versions. Additionally, since the NDC-Flat model in graph-tool also uses the same formula as Eq. (1) for its $-\log p(e|b)$ component, the same argument can be made for NDC-Flat. We focus on DC-Flat for the following experiment.

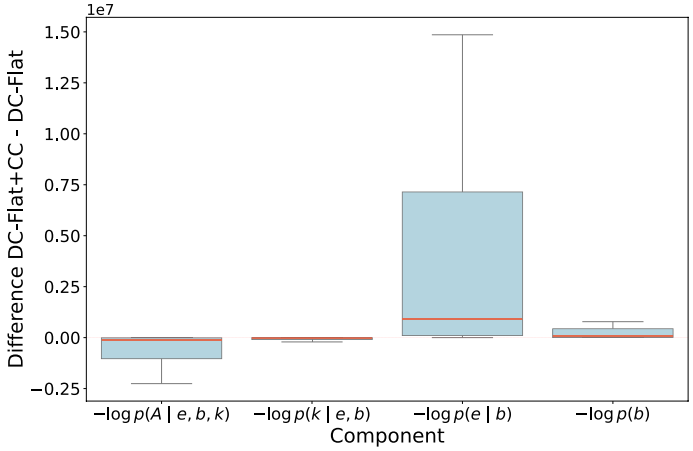


Fig. 4 Experiment 4: DC-Flat+CC to DC-Flat difference for description length components. We show the contribution of $-\log p(e|b)$ term on the description lengths of CC-treated SBM clusterings. Box plots of differences for components of the description length on the 65 networks that select the DC-Flat model for community detection and clusterings obtained by both DC-Flat and DC-Flat+CC for each network. The differences are DC-Flat+CC - DC-Flat. Positive values indicate favoring not having the CC treatment

Table 4 Experiment 4: Components of description lengths on `linux`

Quantity	DC-Flat	DC-Flat+CC	Difference
$-\log p(A\|b,e,k)$	699k	316k	-383k
$-\log p(k\|b,e)$	96k	45k	-51k
$-\log p(e\|b)$	51k	1,585k	1,534k
$-\log p(b)$	147k	257k	110k
DL(*A*, *b*)	993k	2,202k	1,209k

The last row sums up the values in the previous four rows. The difference is DC-Flat+CC - DC-Flat; so, a negative value means favoring DC-Flat with CC treatment, and a positive value means favoring untreated DC-Flat. This table appears as Table 2 in Park et al. (2025)

Graph-tool provides the functionality for computing these components of Eq. (1) separately, which we will use for the following analysis (see the software in Supplementary Materials, Section D). We compute the components of the description length for all networks that select the DC-Flat model, with and without the CC treatment; there are 65 networks (see the full list in Supplementary Materials, Section H.1.). Figure 4 illustrates the distribution of differences between DC-Flat+CC (i.e., the output of the DC-Flat model with CC treatment) and DC-Flat (i.e., the output of the DC-Flat model) for each component on all the networks. Since the difference is DC-Flat+CC - DC-Flat, a positive difference means we do not favor the CC treatment. On all studied networks, the $-\log p(A|b,e,k)$ and $-\log p(k|b,e)$ component prefers connected clusters returned by the CC treatment. In contrast, the $-\log p(b)$ and $-\log p(e|b)$ components penalize the CC treatment, with the $-\log p(e|b)$ component contributing to a larger difference.

We present a specific example of this phenomenon on the real-world network `linux`, detailed in Table 4. The two components $-\log p(A|b,e,k)$ and $-\log p(k|b,e)$ of DC-Flat+CC are lower, indicating an advantage of the CC treatment. However, the two components $-\log p(b)$ and $-\log p(e|b)$ of DC-Flat+CC are higher, especially with the $-\log p(e|b)$ component having a much larger margin; this effectively negates all the advantage. As a result, with a lower description length, the untreated output of the DC-Flat model of SBM is preferred over the same output with CC treatment. Had the

$-\log p(e|b)$ ***not been included*** in the formula for the description length, DC-Flat+CC would have been preferred for having a smaller description length.

Finally, we investigated all non-bipartite networks to see if the CC treatment is preferred when we remove the $-\log p(e|b)$ component. Our investigation shows, for 64 networks out of 65 networks (except `at_migrations`), removing the $-\log p(e|b)$ component will result in a strictly lower description length for the clustering output with CC treatment. Even for `at_migrations`, removing the $-\log p(e|b)$ component will result in the same description length for the clustering output with CC treatment. Thus, the $-\log p(e|b)$ component accounts for 100% of the cases where DC-Flat without CC treatment is preferred over DC-Flat with CC treatment on the non-bipartite real-world networks we studied.

Studying the effect of the priors

The previous discussion used the default configuration where $\beta = 1.0$ and found that certain influential prior components, especially $-\log p(e|b)$, encourage having large but poorly-connected clusters. In the following analysis, we experiment with different weight configurations in order to see if connectivity and clustering accuracy can be improved.

The code for graph-tool allows β to be provided as an input parameter, and also allows each component to be "turned off" individually. For example, turning off the edge count matrix prior will lead to optimizing $\mathrm{DL}(A, b) + \beta \log p(e|b)$ instead of $\mathrm{DL}(A, b)$. Hence, we experiment with varying β and turning off the edge count matrix component. The codes used are given in Supplementary Materials, Section H.2.

Figure 5 shows the effect of different prior weight configurations on the proportion of nodes in different types of clusters detected from real-world networks. Without the $-\log p(e|b)$ component, almost all clusters are singletons. When varying β from 0.0 to 1.0, the proportion of singleton clusters decreases while the proportion of non-singleton clusters that are disconnected increases. We also see a decrease in the number of non-singleton clusters, which may contribute to the proportion of disconnected clusters increasing. However, for all settings of β, the important point is that the incidence of disconnectivity persists.

Supplementary Materials, Fig H shows the effect of prior weight configuration on the community detection accuracy for synthetic networks. When varying β from 0.0 to 1.0 (boxes from left to right), NMI decreases while AMI increases to the peak at $\beta = 0.8$ before dropping slightly. A similar trend to AMI is observed for ARI, but with the peak at $\beta = 0.5$. The trend for NMI is partially explained by NMI's tendency to favor estimated clustering with more clusters, thus favoring clustering with a low node coverage (note that each unclustered node is considered a singleton cluster). On the other hand, the trends for AMI and ARI suggest that there are potential benefits for changing β to a smaller value.

Studying the effect of providing the correct number of clusters

In this section, we experimentally study the frequency of disconnected clusters produced by Chosen-Flat models, when given EC-SBM synthetic networks and told the correct number of clusters; the ability to provide this information is a feature of graph-tool that we take advantage of in this experiment.

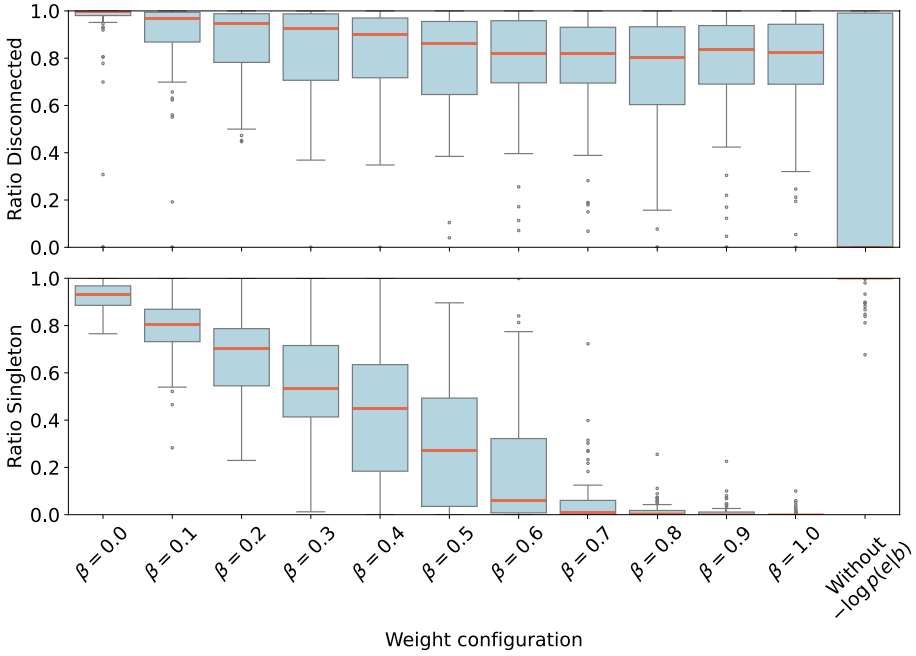


Fig. 5 Experiment 4: Effect of prior weight configuration on connectivity of detected clusters on real-world networks The result is collected from 74 real-world networks. The *x*-axis shows different prior weight configurations, which is either by different values of β or by deactivating the $-\log p(e|b)$ component (right-most box plot). Ratio Disconnected is the proportion of non-singleton clusters that are disconnected. Ratio Singleton is the proportion of singleton clusters out of all clusters. We observed that the frequency of singleton clusters increased as we lowered the influence of the priors (with decreasing β) and was close to 100% when we deactivated the $-\log p(e|b)$ component

We specifically sought to determine whether this knowledge would be sufficient to completely eliminate the appearance of disconnected ground-truth clusters, a feasible objective since, by design, the ground-truth communities within EC-SBM synthetic networks are guaranteed to be connected. If not, then this would suggest that the preference for a small number of clusters (as provided in our theoretical analysis of the description length formula for the DC-SBM model) is not a sufficient explanation for why Flat-DC (and other SBMs) produce disconnected clusters.

Furthermore, even if we find that providing this information to graph-tool does not eliminate disconnected clusters, we wished to determine whether providing the number of clusters reduced the frequency of disconnected clusters, and if so, to what extent. This aspect of the evaluation is motivated by the experimental evidence from Table 3 that Chosen-Flat consistently returns a smaller number of clusters than the ground truth.

This experiment was performed as follows. First, for each synthetic EC-SBM network N with ground-truth clustering, we compute the clustered subnetwork, which is the subnetwork of N induced by the nodes within non-singleton clusters. We let B denote the number of non-singleton clusters. Then, we perform the Chosen-Flat pipeline (i.e., running Flat-DC, Flat-NDC, and Flat-PP and selecting the clustering with the smallest description length) on the clustered subnetwork. We repeat this pipeline, giving graph-tool the value B, i.e., the correct number of non-singleton clusters, and again select from the resultant clusterings the one that has the smallest description length. We refer to this second pipeline as "Chosen-Flat(B)", to indicate that we provided B to graph-tool.

Figure 6 shows that giving the correct number of clusters has a varying impact on the frequency of disconnected clusters that depends on the type of EC-SBM network: those based on the input clustering Leiden-CPM(0.1) or those based on Leiden-Mod.

When computing a clustering on the EC-SBM networks based on Leiden-CPM(0.1), there is a large drop in the median frequency of disconnected clusters, from more than 60% disconnected to less than 5% disconnected. Thus, on this type of EC-SBM network, knowing the correct number of clusters *nearly eliminated* the incidence of disconnected clusters.

Interestingly, we did not see any reduction in disconnected clusters when analyzing EC-SBM networks based on Leiden-Mod. That is, without knowing the true number of communities, the median frequency of disconnected clusters was close to 60%, and this value remained the same given the true number of communities. This indicates that the tendency to produce disconnected clusters *remains very high* for this type of EC-SBM network.

A possible explanation for the difference between network types is that the degree to which Chosen-Flat underestimates the true number of clusters is very large for EC-SBM networks based on Leiden-CPM(0.1) but much smaller for EC-SBM networks based on Leiden-Mod (see Table 3). Even so, it is disappointing that such a large fraction of Chosen-Flat(B) clusters are disconnected in this case.

We now more closely examine results on the Leiden-CPM(0.1) EC-SBM type. A large factor for the reduction in the proportion of disconnected clusters may be that Chosen-Flat(B) typically selected the PP-Flat(B) model (58/74 networks, which is approximately 78.38%), which has a much lower frequency of disconnected clusters compared to other flat models in general, and especially when given the correct number of blocks (see Supplementary Materials, Fig I). Another interesting observation is that Chosen-Flat(B) produces more singleton clusters. This suggests that even when knowing the true number of clusters (which we have shown, in Table 3, to be larger than the estimated number of clusters), Chosen-Flat models will achieve the goal by including singleton clusters with non-singleton clusters.

Note also that by fixing the number of clusters, we effectively remove the $-\log p(e|b)$ component from the description length optimization (from Eq. 1, $-\log p(e|b)$ only

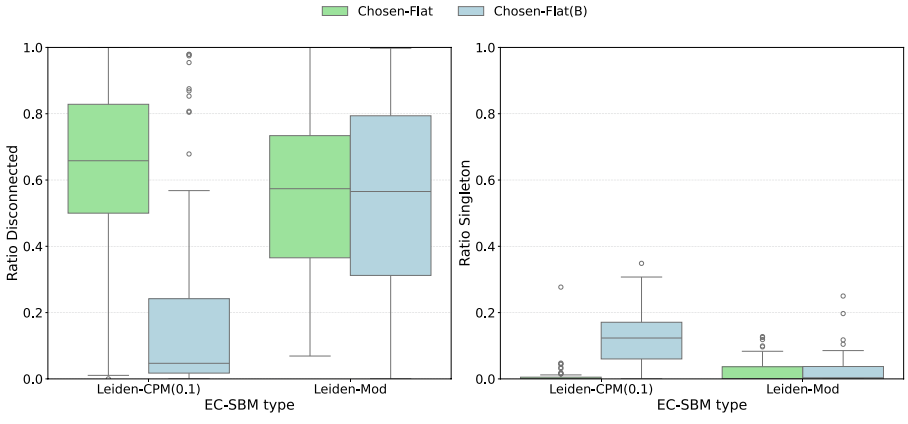


Fig. 6 Experiment 4: Frequency of disconnected clusters of flat SBM models given correct number of blocks in EC-SBM networks. Results are shown for 74 networks. We examine the impact of providing the correct number of blocks, denoted by (B). Ratio Disconnected is the proportion of non-singleton clusters that are disconnected. Ratio Singleton is the proportion of singleton clusters out of all clusters

depends on the number of edges and clusters). However, although with improvements, we are still observing a high frequency of disconnected clusters from DC-Flat and NDC-Flat (see Supplementary Materials, Fig I). This suggests that removing the $-\log p(e|b)$ component from the optimization objective does not solve the connectivity issue.

Experiment 5: Computational performance

We examined the computational performance of the pipelines involving CC and WCC treatments, specifically with respect to runtime and the memory required. There were no failures for the pipelines using CC treatments, but there were a few failures for the pipelines using WCC treatments, all of which occurred during the WCC treatment (i.e., the SBM clustering completed in all analyses). The Chosen-Flat/Nested+WCC pipeline involves computing an SBM clustering under each model (DC-Flat, NDC-Flat, PP-Flat, DC-Nested, and NDC-Nested), and then following with WCC on the clustering that achieved the lowest description length.

Failures on synthetic networks

Recall that for each of the 74 real-world networks we have four EC-SBM networks (each based on a different computed clustering), which results in 296 synthetic networks. For each of these, we have two pipelines: Chosen-Flat and Chosen-Nested, each performed with WCC. Thus, overall we have 592 pipelines using EC-SBM networks that involve WCC. Within the given memory limit of 256GB and a time limit of 3 days, 6 out of 592 runs on these EC-SBM networks did not complete (see Table 5). This represents a failure rate of 1.01%.

For ABCD+o, there are 148 synthetic networks, as we only performed this for two input clusterings (Leiden-Mod and Leiden-CPM(0.1). For each of these we have two pipelines: Chosen-Flat and Chosen-Nested, each followed by WCC. Thus, we have 296 pipelines using ABCD+o that involve WCC. Within the same memory and time limit, 6 of these runs failed, all due to OOM, which is a failure rate of 2.03%.

Given that CC always succeeded but WCC sometimes failed due to memory limits, we conjecture that these failures result when there is a very large and very sparse cluster,

Table 5 Statistics on failed runs

	SBM	Reason	# nodes	c_n	$c_{density}$
EC-SBM network					
Leiden-CPM(0.1) - myspace_aminer	Flat	OOM	853,466	176,295	0.0004
Leiden-CPM(0.01) - myspace_aminer	Flat	OOM	853,466	305,160	0.0001
Leiden-Mod - petster	Flat	TIME	623,748	148,369	0.0014
Leiden-Mod - petster	Nested	SEGF	623,748	139,958	0.0016
Leiden-Mod - berkstan_web	Nested	SEGF	685,230	43,230	0.0071
Leiden-Mod - myspace_aminer	Nested	SEGF	853,466	102,749	0.0011
ABCD+o network					
Leiden-CPM(0.1) - myspace_aminer	Flat	OOM	853,466	840,439	1.1E-5
Leiden-CPM(0.1) - twitter	Flat	OOM	465,017	465,017	7.7E-6
Leiden-CPM(0.1) - hyves	Flat	OOM	1,402,673	1,401,821	2.6E-6
Leiden-CPM(0.1) - hyves	Nested	OOM	1,402,673	1,395,744	2.1E-6
Leiden-Mod - myspace_aminer	Nested	OOM	853,466	205,786	5.3E-5
Leiden-Mod - hyves	Flat	OOM	1,402,673	1,214,910	1.3E-6

For input network *G* with clustering *C*, we let *n* denote the number of nodes. We let c_n denote the number of nodes in the largest cluster and $c_{density}$ denote the density of the largest cluster. We give the reason for failure, noting it is either exceeding the 72 h time limit (TIME), running out of memory (OOM), or segfaulting (SEGF)

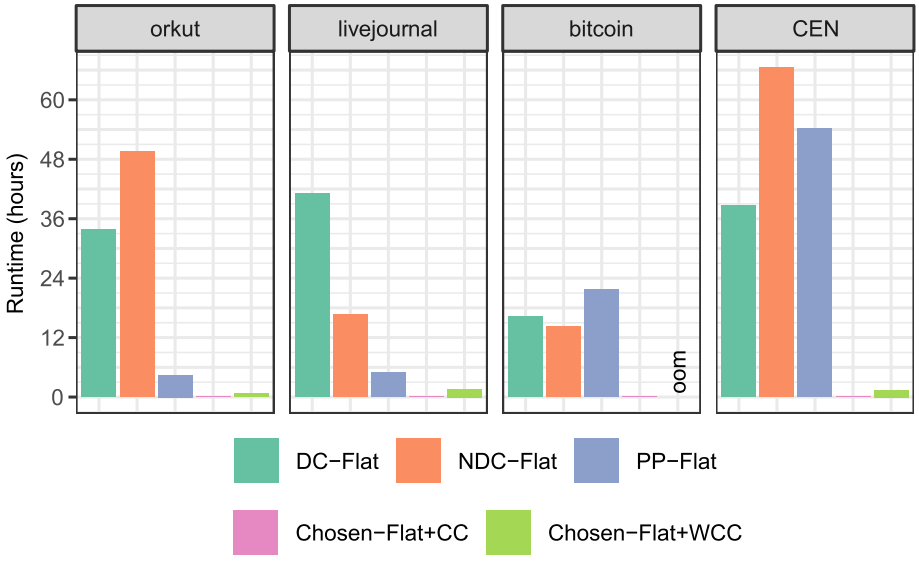


Fig. 7 Experiment 5: Runtime of flat SBM with treatments on large non-bipartite real-world networks For Chosen-Flat+CC and Chosen-Flat+WCC, only the time it took to run the treatment is shown. Chosen-Flat+WCC had an OOM on `bitcoin`. Number of nodes: `orkut` – 3,072,441; `livejournal` – 4,847,571; `bitcoin` – 6,336,770; `CEN` – 13,989,436. The runtime of CC or WCC treatments on SBM clusterings is negligible compared to the runtime of SBM

as this requires repeatedly cutting the cluster into two parts using the code for finding mincuts within VieCut (Henzinger et al. 2018). Specifically, WCC uses the cactus variant of VieCut, which involves a recursive algorithm, and the repeated recursions may be the problem. We therefore checked the size and density of the largest cluster in the relevant clustering (Chosen-Flat or Chosen-Nested), and report this in the last two columns of Table 5. Note that in all of these cases, the largest cluster in the input clustering is very sparse (density at most 0.01). Further research is needed to confirm this hypothesis, but if it turns out to be the case, then replacing the cactus variant with another variant in VieCut is a next step.

Performance on real-world networks

We show the runtime on the four largest real-world networks (see Fig. 7). For this analysis, we limited each run to only a single core (i.e., no parallelization) for both graph-tool's inference and treatments. In general, the WCC treatment was able to complete on nearly every network we analyzed, except for one real-world network (`bitcoin`) due to an out-of-memory (OOM) error.

As shown in Fig. 7, by far the most computationally intensive part was computing the SBM clustering for each of the three models, which took between 5 and 66 h each. Running CC was negligible, completing in minutes, and running WCC finished in under two hours on each network. For example, on the CEN, which has about 14 million nodes, the SBM model with the least runtime was the DC-Flat model, which took 38.7 h. In comparison, WCC processing took 1.4 h. Other SBM models on the CEN were more expensive, with NDC-Flat at 66.5 h and PP-Flat at 54.2 h. Thus, the time it took to cluster the networks using SBM far exceeded the time it took to process those clusterings through the CC and WCC treatments.

Discussion

Incidence of disconnected clusters

A major finding from this study is that alternative models and software for SBM clustering did not succeed in reliably producing connected clusters. For example, we found that PySBM had the same problem with producing disconnected clusters as graph-tool when using the models that both tools enable. Furthermore, although PySBM enables additional models compared to graph-tool, clusterings under these models also had a high frequency of disconnected clusters. Thus, PySBM does not provide a solution to this problem. Furthermore, although the improvement in description length was small, graph-tool was able to produce better (lower) description lengths than PySBM for models that both tools implemented. Together, these observations indicate that the main advantage of PySBM may be the additional models it enables, and that otherwise, graph-tool is a better choice than PySBM for clustering networks using SBMs.

Nevertheless, our study shows that all models within graph-tool produced disconnected clusters (Table 1). Flat models had a slightly higher frequency than nested (i.e., hierarchical models), and DC (degree-corrected) had a slightly higher frequency than NDC (non-degree-corrected). The lowest frequency was found by PP (planted partition), but its mean frequency of disconnected clusters is still 62%. The observation that all models had a high frequency of disconnected clusters suggests that the "resolution limit" exists for all these SBM models.

Because SBM clusterings based on all models had a high frequency of disconnected clusters, we explored whether there are potential benefits in changing the weight configuration to reduce the influence of the prior components of the description length. We reiterate the argument in Park et al. (2025) that some components, especially the prior for the edge count matrix, heavily penalize having a large number of clusters, with the result that graph-tool's SBM models favor fewer rather than more clusters. By reducing the influence of the prior by setting a lower weight for the prior components or removing the edge count matrix prior completely, we observed a tendency to make smaller clusters, to the extent of making each node a singleton cluster. This helps some accuracy metrics (NMI) but hurts others (AMI and ARI). However, there are weight values between 0.0 and 1.0 for the collection of networks that we analyze (i.e., EC-SBM synthetic networks with Leiden-CPM(0.1) input clustering), where overall improvement in clustering accuracy may be obtained; this trend should be investigated in future work.

We also investigated whether the incidence of disconnectivity could be resolved by giving graph-tool's flat SBM models the correct number of clusters. We observed that even with the additional knowledge, the flat models still produce disconnected clusters. Specifically, on EC-SBM synthetic networks of Leiden-Mod type, the effect of knowing the true number of clusters is minimal, possibly because the estimated clustering without that knowledge is already close to the ground truth in the number of clusters. On the other hand, on EC-SBM synthetic networks of Leiden-CPM(0.1) type, the PP-Flat model improved significantly in terms of disconnectivity and was also the one that produced the lowest description length clustering for a majority of analyzed networks.

Impact of CC and WCC

Our study evaluated the impact of post-processing using either CC or WCC, and found that both often but not always improve clustering accuracy on synthetic networks,

especially with respect to the detection of dense ground-truth clusters. The exceptions occurred when close to 100% of the nodes in the network were in very sparse ground-truth clusters, and this reduction in accuracy occurred only when Leiden-Mod was used to provide the parameters to the synthetic network generators (EC-SBM and ABCD+o).

To understand these trends, note that our study shows that SBM clusterings and Leiden-Mod clusterings share several similarities, in particular they both produce community structures where nearly all the nodes are in very sparse clusters. This finding is consistent with the fact that some versions of modularity-optimization and maximum likelihood estimation of some SBMs are equivalent (Bickel and Chen 2009; Newman 2016). Furthermore, prior studies have established that modularity optimization has a tendency to put a very large fraction of the nodes into one large sparse cluster (Leskovec et al. 2010). Thus, this finding is not surprising, given the prior literature.

Given this, we now consider why CC and WCC hurts accuracy for SBM-based community detection when the vast majority of the nodes are in very sparse ground-truth clusters. The reason is basically that SBM clustering is suited to networks with such a high coverage by very sparse ground-truth clusters. Given highly accurate clusters, breaking them up reduces accuracy. In contrast, this reduction in accuracy does not occur when there are sparse ground-truth clusters but they do not cover the vast majority of the nodes. Moreover, we also saw that applying CC or WCC to SBM clusterings of EC-SBM networks where nearly all the nodes are in very sparse clusters had a *variable* impact on ground-truth cluster recovery: it reduced accuracy for recovery of the very sparse ground-truth clusters, but improved accuracy for the denser ground-truth clusters (Fig. 3). This is also consistent with this hypothesis: CC and WCC will hurt accuracy for recovery of very sparse ground-truth clusters, but not for denser clusters.

Conclusions

Summary of findings

Building on the work of Park et al. (2025), this paper presents an extensive study of community detection using Stochastic Block Models (SBMs). Our study significantly expands upon the previous study in several key areas. We evaluated a wider range of SBM software, adding PySBM to the previously tested graph-tool. We also tested more models, including the nested (hierarchical) models within graph-tool. Furthermore, we applied our own modifications, such as different weight configurations and fixing the number of clusters for flat models, to investigate the incidence of disconnectivity. Our analysis used a larger corpus of synthetic networks, which were generated from 74 real-world networks using EC-SBM and ABCD+o.

This study offers new insights into the accuracy and connectivity of clusterings produced by current SBM software and shows the effect of our proposed treatments. Our study revealed that the issue of producing disconnected clusters is present for all tested SBM models, including the ten models explored in PySBM and the five models explored in graph-tool, along with their respective inference algorithms. Although nested SBMs have been hypothesized to mitigate this problem, we found that they do not provide an adequate solution.

The CC (connected components) technique is a simple approach that directly addresses the problem where a clustering has internally disconnected clusters, while the WCC (well-connected clusters) technique addresses the problem of poorly-connected

clusters. Each of these two techniques was demonstrated to improve clustering accuracy, especially for detecting dense clusters, *except* when the vast majority of nodes are in large and very sparse ground-truth clusters. The observed improvement implies that the original SBMs suffer from issues related to the resolution limit, causing them to group multiple dense subgraphs into a single large cluster that is less dense. Under these conditions, CC and WCC achieve better accuracy precisely because they decompose these large clusters.

Finally, our evaluation of the computational performance of CC and WCC treatments on four large networks with up to approximately 14 million nodes shows that they are very fast, running in minutes, and thus a fraction of the hours or days of analysis needed to compute the SBM clusterings. Nevertheless, the WCC software exhibited memory issues, which we are currently addressing.

How to decide if CC and WCC should be used?

A basic question one may ask is when it is desirable to use CC or WCC to post-process an SBM clustering. Given that CC and WCC generally improved accuracy for SBM clusterings whenever the ground-truth community structure did *not* have the vast majority of the nodes in very sparse clusters, one might try to predict, from the empirical SBM clustering, whether the true community structure has that problematic property where nearly all nodes are in very sparse clusters. However, from Fig. 2 and Supplementary Materials, Fig D, we see that the empirical properties of SBM clusterings look very similar for all EC-SBM network types, and they also all resemble the EC-SBM network type based on Leiden-Mod. Thus, at present, it is not clear how we can use empirical properties of the computed SBM clustering to intelligently guess at the nature of the ground-truth clustering.

Given that we currently cannot propose a way to confidently detect the condition of having close to 100% of the nodes in ground-truth communities that are very sparse, perhaps the best decision is not so much based on “what do you think the true community structure is?" but rather “is your goal to recover all communities of any density, or are you willing to lose some communities that are sparse so that you can get better accuracy and recovery of the denser communities?”. For those applications where sparse clusters are not desirable, our study suggests that WCC and CC are appropriate tools for post-processing SBM clusterings. However, for an application where sparse clusters are realistic and needed, then using the SBM clustering without post-processing may be preferred. As for the choice between CC and WCC, when cluster quality is considered, it is clear that CC is a reasonable, and perhaps advisable, step, since otherwise clusters are produced that are internally disconnected; moreover, for those conditions where recovery of sparse clusters is desired and the true community structure may be similar to Leiden-mod, CC has less of a detrimental impact than WCC. Nevertheless, we also saw that WCC improved recovery of dense clusters better than CC. Thus, as with all questions about choice of clustering method, the answer to the question depends on the user’s application and needs (Von Luxburg et al. 2012).

Suggestions for future work

Future research related to SBM clustering should explore modifications to the priors in the description length formulas to investigate whether changes can reduce the tendency

to disconnected clusters while maintaining or improving clustering accuracy. One potential approach is a granular analysis of the effect of individual prior weights, potentially leading to an adaptive scheme where the properties of the input networks are taken into consideration in setting the weights. Another approach is to redesign the priors to explicitly penalize disconnectivity and reward well-connectedness; however, this seems very difficult to achieve. Alternatively, the search strategy used within the search for the SBM with the minimum description length might be modified to ensure that all blocks (i.e., communities) are at least connected, preferably well-connected.

One major direction for future work would reconsider the threshold used for "well-connectedness", since the $\log_{10} n$ threshold may not be optimal, especially for those networks where nearly all nodes are in very sparse clusters, we can consider modifications that might split fewer clusters and hence improve accuracy. One option is to change the threshold from $\log_{10} n$ to a threshold that is easier to meet, even $\log_{10} n - 1$. Indeed, the impact of using other thresholds has not yet been examined in these different post-processing techniques (the Connectivity Modifier (Park et al. 2024a) or the prior study introducing WCC (Park et al. 2025)) and it is possible that changed values could improve accuracy under different conditions. However, we also propose the following approach: to change the definition of the threshold so that instead of it being a function of the number of vertices, it is a function of the properties of the cluster (e.g., number of nodes in the cluster and the degree sequence or just the density), and the expected size of a minimum edge cut for a cluster with those properties. Thus, we will not split the cluster by removing the small edge cut unless the minimum edge cut is much smaller than the expected size of a minimum edge cut.

Other future work should address the memory issues of the WCC software; as WCC depends on VieCut (Henzinger et al. 2018), this issue may be solved through changes to how the minimum cuts are found.

Supplementary Information

The online version contains supplementary material available at https://doi.org/10.1007/s41109-025-00747-2.

Supplementary file 1.

Acknowledgements
The authors thank George Chacko for helpful feedback.

Author contributions
TVL provided the EC-SBM networks, generated the ABCD+o networks, and evaluated graph-tool clustering methods using both real and synthetic networks, analyzed the data, and wrote the first draft of the manuscript. MP developed the codes for CC and WCC and assisted in writing the first draft. IC evaluated PySBM and SBMs within graph-tool using both real and EC-SBM synthetic networks, analyzed the data, and assisted in writing the first draft. TW supervised the research, evaluated experimental results, and edited the drafts of the manuscript.

Funding
This work was supported in part by the Illinois-Insper partnership and the US National Science Foundation grant 2402559 (to TW).
The authors thank the Illinois Computes Program for allocations of cluster computing time.

Data availability
The real-world networks on which the analyses are based are already in the public domain. The EC-SBM networks are available at Vu-Le, Park, Chen, and Warnow (2025). The ABCD+o networks generated for this paper are available at Vu-Le, Chacko, and Warnow (2025). The software used is in the public domain. The analysis scripts are available at Chen et al. (2025). The commands used to perform analyses are provided in the Supplementary Materials.

Declarations

Competing interests
The authors declare that they have no Conflict of interest.

Received: 5 August 2025 / Accepted: 12 October 2025
Published online: 28 November 2025

References

Abbe E (2018) Community detection and stochastic block models: recent developments. J Mach Learn Res 18(177):1–86
Anne L, Vu-Le T-A, Park M, Warnow T, Chacko G (2025) RECCS: realistic cluster connectivity simulator for synthetic network generation. Adv Complex Syst. https://doi.org/10.1142/s0219525925400041
Bickel PJ, Chen A (2009) A nonparametric view of network models and Newman-Girvan and other modularities. Proc Natl Acad Sci 106(50):21068–21073
Blondel VD, Guillaume J-L, Lambiotte R, Lefebvre E (2008) Fast unfolding of communities in large networks. J Stat Mech: Theory Exp 2008(10):10008
Chen I, Vu-Le T-A, Park M Analysis scripts for "Using Stochastic Block Models for Community Detection". https://github.com/illinois-or-research-analytics/network-analysis-code
Cherifi H, Palla G, Szymanski BK, Lu X (2019) On community structure in complex networks: challenges and opportunities. Appl Netw Sci 4(1):1–35
Dongen SV (2008) Graph clustering via a discrete uncoupling process. SIAM J Matrix Anal Appl 30(1):121–141
Fortunato S (2010) Community detection in graphs. Phys Rep 486(3–5):75–174
Fortunato S, Hric D (2016) Community detection in networks: a user guide. Phys Rep 659:1–44. https://doi.org/10.1016/j.physrep.2016.09.002
Funke T, Becker T (2019a) Stochastic block models: a comparison of variants and inference methods. PLoS One 14(4):1–40. https://doi.org/10.1371/journal.pone.0215296
Funke T, Becker T (2019b) PySBM. https://github.com/funket/pysbm
Harenberg S, Bello G, Gjeltema L, Ranshous S, Harlalka J, Seay R, Padmanabhan K, Samatova N (2014) Community detection in large-scale networks: a survey and empirical evaluation. Wiley Interdisciplinary Rev Comput Stat 6(6):426–439
Henzinger M, Noe A, Schulz C, Strash D (2018) Practical minimum cut algorithms. ACM J Exp Algorithmics 23:1–22
Holland PW, Laskey KB, Leinhardt S (1983) Stochastic blockmodels: first steps. Social Netw 5(2):109–137
Hubert L, Arabie P (1985) Comparing partitions. J Classif 2:193–218
Javed MA, Younis MS, Latif S, Qadir J, Baig A (2018) Community detection in networks: a multidisciplinary review. J Netw Comput Appl 108:87–111
Jin D, Yu Z, Jiao P, Pan S, He D, Wu J, Yu PS, Zhang W (2023) A survey of community detection approaches: from statistical modeling to deep learning. IEEE Trans Knowl Data Eng 35(2):1149–1170. https://doi.org/10.1109/TKDE.2021.3104155
Kamiński B, Prałat P, Théberge F (2023) Artificial benchmark for community detection with outliers (ABCD+o). Appl Netw Sci 8(1):25
Kannan R, Vempala S, Vetta A (2004) On clusterings: good, bad and spectral. J ACM (JACM) 51(3):497–515
Karrer B, Newman ME (2011) Stochastic blockmodels and community structure in networks. Phys Rev E: Stat, Nonlin, Soft Matter Phys 83(1):016107
Karypis G, Kumar V (1998) A fast and high quality multilevel scheme for partitioning irregular graphs. SIAM J Sci Comput 20(1):359–392
Kernighan BW, Lin S (1970) An efficient heuristic procedure for partitioning graphs. Bell Syst Techn J 49(2):291–307. https://doi.org/10.1002/j.1538-7305.1970.tb01770.x
Lancichinetti A, Fortunato S, Radicchi F (2008) Benchmark graphs for testing community detection algorithms. Phys Rev E 78(4):046110
Lee C, Wilkinson DJ (2019) A review of stochastic block models and extensions for graph clustering. Appl Netw Sci 4(1):122. https://doi.org/10.1007/s41109-019-0232-2
Leskovec J, Lang KJ, Mahoney M (2010) Empirical comparison of algorithms for network community detection. In: Proceedings of the 19th International Conference on World Wide Web, pp. 631–640
Liu X, Song W, Musial K, Li Y, Zhao X, Yang B (2025) Stochastic block models for complex network analysis: a survey. ACM Trans Knowl Discov Data 19(3):1–35
Luxburg Von U, Williamson RC, Guyon I (2012) Clustering: Science or art? In: Proceedings of ICML Workshop on Unsupervised and Transfer Learning, pp. 65–79. JMLR Workshop and Conference Proceedings
Newman ME (2016) Equivalence between modularity optimization and maximum likelihood methods for community detection. Phys Rev E 94(5):052315
Park M, Tabatabaee Y, Ramavarapu V, Liu B, Pailodi VK, Ramachandran R, Korobskiy D, Ayres F, Chacko G, Warnow T (2024) Well-connectedness and community detection. PLOS Complex Syst 1(3):0000009
Park M, Feng DW, Digra S, Vu-Le T-A, Chacko G, Warnow T (2025) Improved community detection using stochastic block models. In: Cherifi H, Donduran M, Rocha LM, Cherifi C, Varol O (eds) Complex Netw Their Appl XIII. Springer, Cham, pp 103–114
Peixoto TP (2014) The graph-tool python library. figshare, http://figshare.com/articles/graph_tool/1164194
Peixoto TP (2019) In: Doreian, P., Batagelj, V., Ferligoj, A. (eds.) Bayesian Stochastic Blockmodeling, pp. 289–332. John Wiley & Sons, Ltd, Hoboken, NJ. Chap. 11. https://doi.org/10.1002/9781119483298.ch11
Peixoto TP (2020) The Netzschleuder network catalogue and repository. https://networks.skewed.de
Peixoto TP (2013) Parsimonious module inference in large networks. Phys Rev Lett 110:148701. https://doi.org/10.1103/PhysRevLett.110.148701
Peixoto TP (2014) Efficient Monte Carlo and greedy heuristic for the inference of stochastic block models. Phys Rev E 89(1):012804
Rosvall M, Axelsson D, Bergstrom CT (2009) The map equation. Eur Phys J Special Topics 178(1):13–23
Thomas M, Joy AT (2006) Elements of Information Theory. Wiley-Interscience, Hoboken, NJ
Traag V (2019) leidenalg. https://github.com/vtraag/leidenalg
Traag VA, Waltman L, Van Eck NJ (2019) From Louvain to Leiden: guaranteeing well-connected communities. Sci Rep 9(1):1–12

Vinh NX, Epps J, Bailey J (2009) Information theoretic measures for clusterings comparison: is a correction for chance necessary? In: Proceedings of the 26th Annual International Conference on Machine Learning, pp. 1073–1080
Vu-Le T-A, Chacko G, Warnow T. EC-SBM Benchmark Networks. https://doi.org/10.13012/B2IDB-3284069_V1
Vu-Le T-A, Park M, Chen I, Warnow T (2025) Data for "Using Stochastic Block Models for Community Detection". University of Illinois Urbana-Champaign. https://doi.org/10.13012/B2IDB-3421614_V1
Vu-Le T-A, Anne L, Chacko G, Warnow T (2025) EC-SBM synthetic network generator. Appl Netw Sci 10(1):15
Wedell E, Park M, Korobskiy D, Warnow T, Chacko G (2022) Center-periphery structure in research communities. Quantitative Sci Stud 3(1):289–314
Yang J, Leskovec J (2013) Defining and evaluating network communities based on ground-truth. Knowl Inf Syst 42(1):181–213
Young J-G, Desrosiers P, Hébert-Dufresne L, Laurence E, Dubé LJ (2017) Finite-size analysis of the detectability limit of the stochastic block model. Phys Rev E 95(6):062304
Zhang AY (2024) Fundamental limits of spectral clustering in stochastic block models. IEEE Transactions on Information Theory. https://doi.org/10.1109/TIT.2024.3425581
Zhang L (2023) Realistic constraints, model selection, and detectability of modular network structures. PhD thesis, University of Bath
Zhang L, Peixoto TP (2020) Statistical inference of assortative community structures. Phys Rev Res 2(4):043271
Zhao Y (2017) A survey on theoretical advances of community detection in networks. Wiley Interdisciplinary Rev Comput Stat 9(5):1403

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.